



PARISH DIGITAL PLATFORM · PROJECT BLUEPRINT

# St. Michael Madende Parish Website & Member Portal

A complete technical and functional specification for St. Michael Madende Parish's full-stack digital platform — built with Django REST Framework and React, styled with Tailwind CSS — designed to serve parishioners, visitors, clergy, and administrators alike.

DJANGO REST FRAMEWORK

REACT + VITE

TAILWIND CSS

POSTGRESQL

**Prepared for:** St. Michael Madende Parish, via jnolly IT Solutions

**Prepared by:** Japeth Anold Dindi

**Document type:** Full-Stack Project Specification & Documentation

**Version:** 2.0 (Revised — Tailwind Edition)

# Table of Contents

PART I — FOUNDATIONS

01 Executive Summary & Project Goals

02 Recommended Technology Stack

03 System Architecture

04 Design System & Tailwind Configuration

PART II — SITE STRUCTURE & PAGES

05 Navigation & Sitemap

06 Homepage

07 About the Church & Clergy

08 Mass Schedule & Liturgical Calendar

09 Sacraments & Online Registration

10 Events, News & Church Calendar

11 Gallery, Homilies & Live Streaming

12 Daily Readings & Prayer Wall

13 Online Giving & Payments

14 Ministries, Staff Directory & Contact

15 FAQ, Search & Notifications

PART III — PLATFORM & ENGINEERING

16 Data Model & Database Schema

17 REST API Design

18 Authentication, Roles & Permissions

19 Admin Dashboard & Member Portal

20 Security & Data Protection

21 Accessibility & Internationalization

22 Performance, SEO & PWA

23 Testing Strategy

24 DevOps, CI/CD & Deployment

PART IV — DELIVERY

25 Advanced & Differentiating Features

**26** Suggested Folder Structure

---

**27** Delivery Roadmap & Sprint Plan

---

**28** Closing Notes

---

# Executive Summary & Project Goals

For St. Michael Madende Parish, a website is the digital front door of the community. It should do far more than display information beautifully — it must help parishioners stay spiritually and administratively connected, give visitors a clear path to belong, and give the parish office a real operational tool. Because this is also a portfolio-grade build, the project is scoped to demonstrate production-level full-stack engineering: authentication, RBAC, payments, real-time features, search, scheduling, and a polished, accessible UI.

## For Parishioners

- Instant access to Mass times, readings, and sacraments
- Register for events, classes, and sacrament preparation online
- Give securely via M-Pesa, card, or bank transfer
- Submit prayer requests and follow parish news

## For Visitors & Newcomers

- Clear, welcoming introduction to the parish and its leadership
- Directions, service times, and "how do I join" guidance
- Watch live Mass or catch up on recent homilies

## For Clergy & Ministry Leaders

- Publish homilies, announcements, and homily notes directly
- Manage ministry membership and activities
- View and respond to prayer requests and registrations

## For the Parish Office (Admin)

- One dashboard for events, donations, sacraments, and users
- Automated notifications and confirmation emails/SMS
- Reporting on attendance, giving, and registrations

**What's new in this revision:** the frontend stack now uses **Tailwind CSS** in place of Bootstrap for full design control and a lighter bundle; the platform adds real-time features (live notifications, chat-style prayer requests), a proper data model and API contract, a security & accessibility chapter, automated testing, and a phased delivery roadmap — turning the original page-by-page brief into an implementation-ready specification.

# Recommended Technology Stack

Bootstrap has been replaced with Tailwind CSS throughout — utility-first styling gives finer control over the charcoal/white/red/blue parish theme without fighting component overrides, and pairs naturally with React + Vite.

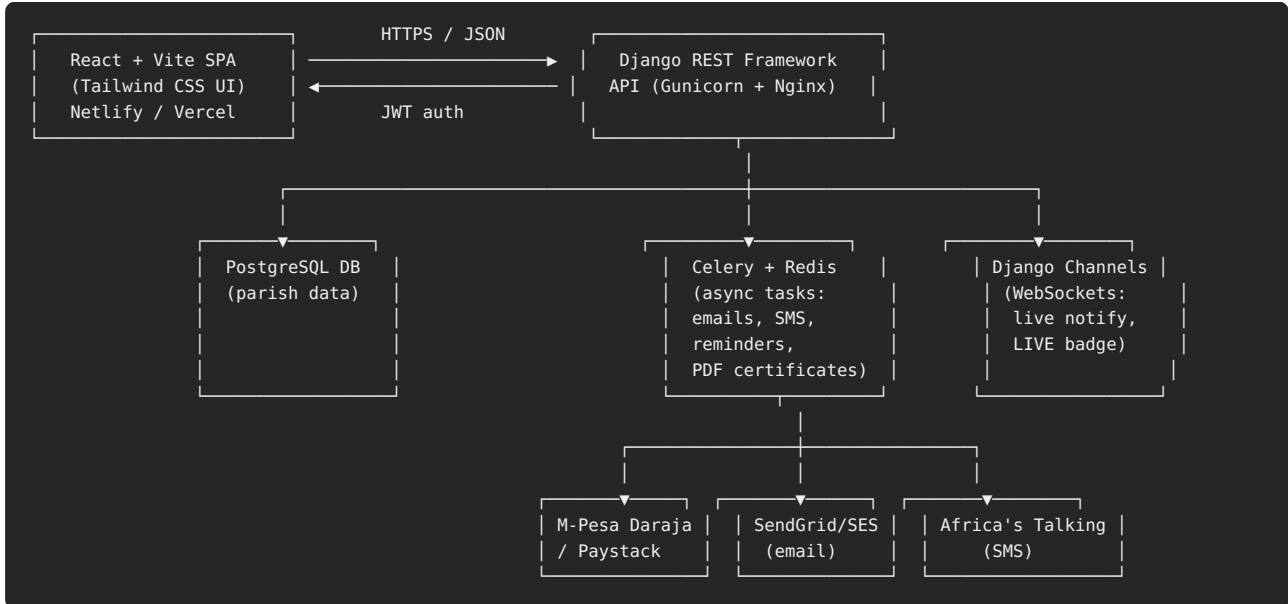
| Layer              | Technology                                                                      | Why                                                                                                                                                    |
|--------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend Framework  | Django 5 + Django REST Framework                                                | Batteries-included ORM, admin, auth, and a mature REST layer for the mobile-friendly API                                                               |
| Database           | PostgreSQL (SQLite for local dev)                                               | JSONB for flexible content blocks, full-text search, strong relational integrity                                                                       |
| Async / Real-time  | Django Channels + Redis + Celery                                                | Live-stream "LIVE NOW" badge, live notifications, background email/SMS, scheduled reminders                                                            |
| Frontend Framework | React 18 + Vite                                                                 | Fast dev server, component-driven UI, easy code-splitting per page                                                                                     |
| Styling            | <b>Tailwind CSS 3</b> + Headless UI + Framer Motion                             | Utility-first styling matched to the parish design tokens; Headless UI for accessible menus/modals; Framer Motion for subtle scroll/entrance animation |
| Icons              | Lucide React (replaces Font Awesome)                                            | Tree-shakeable SVG icon set that plays well with Tailwind sizing utilities                                                                             |
| Rich Text          | TipTap editor (replaces CKEditor)                                               | Headless, React-native rich text editor for homilies, news, and sacrament pages                                                                        |
| Auth               | Django + SimpleJWT, refresh-token rotation                                      | Stateless JWT auth for the API, with role-based permission classes                                                                                     |
| Payments           | M-Pesa Daraja (STK Push) + Paystack/Flutterwave (cards)                         | Local mobile money plus international card support for the diaspora                                                                                    |
| Media Storage      | Cloudinary or AWS S3 (django-storages)                                          | Offloads gallery photos, homily audio/video, and PDFs from the app server                                                                              |
| Search             | PostgreSQL full-text search (or Elasticsearch at scale)                         | Unified search across news, events, priests, ministries, homilies                                                                                      |
| Notifications      | Django Channels (in-app), Celery + SendGrid/SES (email), Africa's Talking (SMS) | Multi-channel reminders for Mass, events, and registrations                                                                                            |
| Testing            | Pytest + pytest-django, React Testing Library, Playwright                       | Unit, API, and end-to-end coverage                                                                                                                     |
| CI/CD              | GitHub Actions                                                                  | Automated lint/test on push, build & deploy on merge to main                                                                                           |
| Deployment         | Docker + Gunicorn + Nginx, hosted on Railway/Render or a VPS                    | Reproducible builds; easy horizontal scaling of the web and worker services                                                                            |

## Why drop Bootstrap for Tailwind

- **No fighting default component styles** — Bootstrap's `.btn`, `.card`, `.navbar` come with opinions that need overriding to hit a bespoke, reverent aesthetic; Tailwind starts from nothing and builds up.
- **Smaller shipped CSS** — Tailwind's JIT compiler only ships classes actually used, versus Bootstrap's larger fixed bundle.
- **Design-token driven** — the charcoal/white/red/blue palette, spacing, and radii become first-class values in `tailwind.config.js`, reused consistently across every component and easy for future contributors (e.g. other Moi University students) to pick up.
- **Dark mode "for free"** — Tailwind's `dark:` variant makes the requested dark-mode feature a straightforward addition rather than a second stylesheet.

# System Architecture

A decoupled architecture: React/Vite SPA talking to a Django REST API, with Celery workers and Channels handling anything asynchronous or real-time.



## Key architectural decisions

- **API-first backend** — the same DRF API serves the public website, the member portal, and (optionally) a future mobile app, so nothing is built twice.
- **Async by default for anything slow** — sending SMS/email, generating a baptism certificate PDF, or processing a donation webhook never blocks the request/response cycle; it's handed to Celery.
- **WebSockets only where they earn their keep** — the "LIVE NOW" badge, live prayer-wall updates, and admin notification bell use Channels; everything else stays simple request/response.
- **Media never touches the app server's disk** — gallery images, homily audio, and bulletins are uploaded straight to Cloudinary/S3 to keep the Django containers stateless and easy to scale.

# Design System & Tailwind Configuration

A bold, high-contrast palette — charcoal gray and white carry the page as neutral ground, with pure red and pure blue reserved as clear, confident accent colors for actions, links, and section markers.



```
// tailwind.config.js
module.exports = {
  darkMode: 'class',
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: {
    extend: {
      colors: {
        charcoal: { DEFAULT: '#333333', 700: '#4D4D4D', 900: '#1A1A1A' },
        white: 'FFFFFF',
        red: { DEFAULT: '#FF0000', 700: '#CC0000' },
        blue: { DEFAULT: '#0000FF', 700: '#0000CC' },
        neutral: { 100: '#F5F5F5', 200: '#E2E2E2' },
      },
      fontFamily: {
        display: ['"Playfair Display"', 'serif'],
        sans: ['"DM Sans"', 'sans-serif'],
        mono: ['"JetBrains Mono"', 'monospace'],
      },
      borderRadius: { xl2: '1.25rem' },
    },
  },
  plugins: [require('@tailwindcss/forms'), require('@tailwindcss/typography')],
}
```

## Typography & component conventions

- **Headings:** Playfair Display (serif) for H1/H2 on a white ground; DM Sans for body and UI chrome.
- **Buttons:** primary = bg-red hover:bg-red-700 text-white, secondary = border border-blue text-blue hover:bg-blue hover:text-white, on dark surfaces = bg-white text-charcoal-900.
- **Cards:** rounded-xl2 shadow-sm bg-white border border-charcoal/10 dark:bg-charcoal-900 dark:border-white/10 for a consistent, crisp elevation across event, ministry, and clergy cards.
- **Links & focus states:** links use text-blue with a hover:underline; active/current-page indicators and required-field markers use text-red so the two accents stay functionally distinct rather than interchangeable.
- **Dark mode:** toggling the class strategy flips charcoal-900/white backgrounds and text automatically across every component, keeping red and blue as the constant accent pair in both modes — no duplicate stylesheet needed.

# Navigation & Sitemap

| Top-level        | Children                                                                                                            |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Home             | —                                                                                                                   |
| About Us         | Our History, Mission & Vision, Patron Saint, Diocese, Leadership                                                    |
| Our Parish       | Clergy, Parish Council, Staff Directory                                                                             |
| Mass Schedule    | Weekly Times, Holy Days, Confession, Adoration, Rosary                                                              |
| Sacraments       | Baptism, Confirmation, Eucharist, Reconciliation, Matrimony, Holy Orders, Anointing — each with online registration |
| Ministries       | Choir, Youth, CMA, CWA, Legion of Mary, Altar Servers, SCCs                                                         |
| Events           | Upcoming Events, Church Calendar, Retreats & Pilgrimages                                                            |
| Gallery          | Photos, Videos — filterable by occasion                                                                             |
| Sermons/Homilies | By Priest, by Date, by Gospel Reading                                                                               |
| News             | Announcements, Parish Bulletin (PDF archive)                                                                        |
| Donate           | Give Now, Building Fund, Charity, Thanksgiving                                                                      |
| Contact Us       | Map, Office Hours, Contact Form, FAQ                                                                                |
| Member Login     | Member Portal / Admin Dashboard (role-based)                                                                        |



# Homepage

---

The homepage is the parish's first impression — it must load fast, read clearly on a phone, and get a first-time visitor to a next step within seconds.

## Hero Section

- Church name, logo, and a rotating hero image/video of the church
- Warm welcome message from the parish priest (short quote + "Read More")
- Sticky "Today's Mass" strip: next Mass time + countdown

## Calls to Action

- Join Our Parish · View Mass Times · Donate · Watch Live Mass
- A persistent "LIVE NOW" pill replaces the CTA row automatically during a stream

## Content Blocks

- Upcoming events carousel (next 3–5 events)
- Latest announcements (3 most recent, "View all" link)
- Daily Gospel snippet with a link to the full reading

## Trust & Social Proof

- Ministry highlight strip (logos/photos of active ministries)
- Newsletter signup and social media links in a closing band

# About the Church & Clergy

## About the Church

- Parish history, mission, and vision, plus a dedicated page for the patron, **St. Michael the Archangel** — his feast day (29 September) auto-highlighted on the liturgical calendar each year
- Parish boundaries and diocese affiliation, with an embedded map of Madende and the surrounding boundary
- Church leadership overview linking through to full clergy and parish council profiles

## Meet the Clergy

Cards for the Parish Priest, Assistant Priest(s), Deacons, Religious Sisters, Catechists, and Parish Council, each with photo, name, position, short biography, and contact — built as a reusable `<ClergyCard />` React component so the same design serves every role.

| Field           | Type                     | Notes                                                           |
|-----------------|--------------------------|-----------------------------------------------------------------|
| Photo           | Image                    | Cropped to a consistent 4:5 ratio via Cloudinary transformation |
| Name & Title    | Text                     | e.g. "Fr. John Otieno — Parish Priest"                          |
| Biography       | Rich text                | Short (150–250 word) formation and ministry history             |
| Contact         | Email / office extension | Optional — priest can choose to hide direct contact             |
| Ordination date | Date                     | Powers an optional "years of service" badge                     |

# Mass Schedule & Liturgical Calendar

| Day       | Morning | Evening            |
|-----------|---------|--------------------|
| Monday    | 6:30 AM | 5:30 PM            |
| Tuesday   | 6:30 AM | 5:30 PM            |
| Wednesday | 6:30 AM | 5:30 PM            |
| Thursday  | 6:30 AM | 5:30 PM            |
| Friday    | 6:30 AM | 5:30 PM            |
| Saturday  | 7:00 AM | 5:30 PM            |
| Sunday    | 7:00 AM | 9:00 AM / 11:00 AM |

- Holy Day of Obligation schedules, published automatically ahead of the date
- Confession times, Eucharistic Adoration windows, and Rosary schedule
- **New:** a liturgical calendar widget showing the current season (Advent, Lent, Ordinary Time), the Saint of the Day, and feast days — sourced from a small internal liturgical-calendar table or a calendar API
- **New:** "Add to Calendar" (.ics) button on every Mass time and feast day

# Sacraments & Online Registration

Each sacrament — Baptism, Confirmation, Eucharist, Reconciliation, Matrimony, Holy Orders, Anointing of the Sick — gets its own page with description, requirements, documents needed, an FAQ, and a registration button wired to the same underlying registration workflow.

## Sacrament page anatomy

### Description & Requirements

Plain-language explanation, age/eligibility requirements, and required documents (e.g. birth certificate for Baptism, baptismal certificates for Matrimony).

### Registration Button

Opens a multi-step form (Baptism, Marriage Prep, Confirmation Classes, RCIA, Catechism) that creates a `SacramentRegistration` record and notifies the admin.

### Document Upload

Applicants can attach scanned documents directly (e.g. ID, prior sacrament certificates) — stored securely and visible only to admins/clergy.

### Status Tracking

Registrants can log in to see their application status: *Submitted* → *Under Review* → *Scheduled* → *Completed*.

## Online Sacrament Registration Workflow

1. Visitor/member selects a sacrament and fills the registration form
2. System validates required fields and documents, then creates a pending registration
3. Admin/clergy receive an in-app + email notification
4. Admin approves, requests more info, or schedules a date
5. Applicant is notified automatically at each status change (email/SMS)
6. On completion, an optional certificate PDF is generated and made downloadable from the member portal

# Events, News & Church Calendar

---

## Church Calendar

A unified calendar (month/week/agenda views, built with a component such as `react-big-calendar` styled in Tailwind) showing feast days, weddings, funerals, parish meetings, youth activities, choir practice, pilgrimages, and retreats — filterable by category and exportable per-event as .ics.

## Events Page

- Banner image, date, time, venue, description, and a Register button per event
- Capacity limits with automatic waitlisting once an event is full
- QR-code check-in at the venue (ties into the Advanced Features chapter)

## News & Announcements

A blog-style, tag-filterable feed (Christmas Program, Easter Schedule, Parish Mission, Youth Camp, Bishop's Visit, etc.) with rich text via TipTap, featured images, and an "Emergency Announcement" banner type that pins above all other content site-wide until dismissed or expired.

# Gallery, Homilies & Live Streaming

---

## Gallery

Photos and videos organized into albums (Easter, Christmas, Weddings, Youth Events, Charity Work, Pilgrimages) with a lightweight lightbox, lazy-loaded thumbnails, and Cloudinary-driven responsive image sizes so a 6MB photo never ships to a phone in full resolution.

## Homilies

- Upload video, audio, or PDF notes per homily
- Searchable/filterable by priest, date, and Gospel reading
- Inline audio player with playback-speed control for longer homilies
- Optional auto-generated transcript (Whisper API) for accessibility and search

## Live Streaming

- Embedded YouTube Live and Facebook Live players, switchable by admin
- Site-wide "LIVE NOW" badge (pushed via WebSocket/Channels) appears in the nav and homepage the moment a stream starts
- Automatic fallback to "Watch the most recent Mass" when nothing is currently live

# Daily Readings & Prayer Wall

---

## Daily Readings

First Reading, Responsorial Psalm, Gospel, and an optional short reflection, refreshed automatically each day (via a scheduled Celery task pulling from a liturgical readings API or an internal dataset) and cached so the homepage never waits on an external call.

## Prayer Request

- Simple form: Name, Email, Prayer Request, Private/Public toggle
- Private requests are visible only to admins/clergy in the dashboard
- **New — Prayer Intention Wall:** public requests appear (moderated) on a community wall where other parishioners can tap "Praying for you" — a lightweight, comment-free way to show solidarity without turning it into a social feed
- Basic profanity/spam filtering before anything reaches the public wall

## Online Giving & Payments

| Method        | Provider                 | Use case                                                           |
|---------------|--------------------------|--------------------------------------------------------------------|
| Mobile Money  | M-Pesa Daraja (STK Push) | Primary local giving channel — tithes, thanksgiving, building fund |
| Card          | Paystack or Flutterwave  | Diaspora and card-preferring givers                                |
| Bank Transfer | Manual reference number  | Larger one-off gifts (e.g. building fund pledges)                  |

- Designated giving categories: Parish Projects, Building Fund, Charity, Thanksgiving Offerings, Special Collections
- Instant e-receipt by email/SMS on successful payment (via Celery + webhook confirmation)
- Recurring giving (weekly/monthly standing "pledges") with reminders before each cycle
- Giving history visible to the donor in the member portal; downloadable annual giving statement (PDF) for tax/record purposes
- **New:** QR codes at the physical church entrance and on printed bulletins linking straight to the Donate page for a specific fund



# Ministries, Staff Directory & Contact

---

## Ministries

Choir, Youth, CMA, CWA, Legion of Mary, Altar Servers, Catholic Men, Catholic Women, Small Christian Communities — each with a leader, description, activity calendar feed, and a "Join" button that creates a `MinistryMembershipRequest` for leader approval.

## Parish Staff Directory

Searchable by name, ministry, or office — pulls from the same underlying `People` model used for clergy, so a person moving from "Ministry Leader" to "Parish Council" doesn't require re-entering their profile.

## Contact Page

- Address, email, phone, office hours, interactive map (Google Maps embed or Leaflet/OpenStreetMap)
- Contact form routed to the correct office/ministry inbox by category dropdown
- Direct WhatsApp Business link for quick pastoral queries, where the parish opts in

# FAQ, Search & Notifications

---

## Frequently Asked Questions

Grouped by topic (Mass & Sacraments, Giving, Events, Membership) with an accordion UI and searchable by keyword; content is admin-editable rather than hardcoded so the office can add questions without a developer.

## Site-wide Search

A single search bar (⌘K / Ctrl+K shortcut) queries News, Events, Priests, Ministries, and Homilies simultaneously, grouped by category in the results dropdown — backed by PostgreSQL full-text search initially, upgradeable to Elasticsearch if the content volume grows.

## Notifications

| Channel                  | Trigger examples                                                                      |
|--------------------------|---------------------------------------------------------------------------------------|
| Email                    | Registration confirmation, sacrament status updates, giving receipts, weekly bulletin |
| SMS (Africa's Talking)   | Mass reminders, event reminders, feast-day greetings                                  |
| Push / In-app (Channels) | Admin: new prayer request, new registration; Member: "Your event starts in 1 hour"    |

All notification preferences are user-controlled from the member portal (opt in/out per channel and per category) to respect people's inboxes and avoid notification fatigue.

# Data Model & Database Schema

Core Django apps and their primary models — kept intentionally modular so each area (sacraments, giving, events) can be built and tested independently.

| App           | Key Models                                                       | Notes                                                                                 |
|---------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| accounts      | User, Profile, Role                                              | Custom user model; Role is one of Admin, Clergy, MinistryLeader, Parishioner, Visitor |
| parish        | ChurchInfo, Diocese, ParishBoundary                              | Singleton-style config for church identity, mission, patron saint                     |
| clergy        | Clergy, ParishCouncilMember, StaffMember                         | Shared "Person" profile reused across About, Staff Directory, Ministries              |
| liturgy       | MassTime, HolyDay, LiturgicalSeason, DailyReading, SaintOfTheDay | Powers Mass Schedule, calendar widget, and daily readings                             |
| sacraments    | SacramentType, SacramentRegistration, RegistrationDocument       | Status field: submitted / under_review / scheduled / completed / rejected             |
| events        | Event, EventRegistration, CalendarEntry                          | Capacity, waitlist flag, QR check-in token                                            |
| news          | Announcement, Bulletin (PDF)                                     | Announcement has is_emergency boolean for the sitewide banner                         |
| media         | GalleryAlbum, GalleryItem, Homily                                | Homily links video/audio/PDF plus optional transcript text                            |
| streaming     | LiveStream                                                       | Single active-stream flag drives the "LIVE NOW" badge                                 |
| prayer        | PrayerRequest, PrayerSupport                                     | PrayerSupport = lightweight "praying for you" tap, one per user per request           |
| giving        | Fund, Donation, RecurringPledge                                  | Donation stores provider, reference, status, receipt_sent flag                        |
| ministries    | Ministry, MinistryMembership                                     | MinistryMembership.status: pending / approved                                         |
| notifications | NotificationPreference, NotificationLog                          | One preference row per user per channel/category pair                                 |

```
class SacramentRegistration(models.Model):
    STATUS_CHOICES = [
        ("submitted", "Submitted"), ("under_review", "Under Review"),
        ("scheduled", "Scheduled"), ("completed", "Completed"),
        ("rejected", "Rejected"),
    ]
    applicant = models.ForeignKey(User, on_delete=models.CASCADE)
    sacrament = models.ForeignKey(SacramentType, on_delete=models.PROTECT)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default="submitted")
    scheduled_date = models.DateField(null=True, blank=True)
    notes = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

# REST API Design

Versioned, resource-oriented endpoints under `/api/v1/`, documented automatically with drf-spectacular (OpenAPI/Swagger) so the frontend team (or Japeth's future self) always has an accurate contract.

| Endpoint                                            | Methods                 | Purpose                                                                       |
|-----------------------------------------------------|-------------------------|-------------------------------------------------------------------------------|
| <code>/api/v1/auth/token/, /token/refresh/</code>   | POST                    | JWT login & refresh                                                           |
| <code>/api/v1/mass-times/</code>                    | GET, (Admin) POST/PUT   | Weekly Mass schedule + Holy Days                                              |
| <code>/api/v1/sacraments/{type}/register/</code>    | POST                    | Submit a sacrament registration                                               |
| <code>/api/v1/sacraments/registrations/{id}/</code> | GET, PATCH              | View/update status (role-restricted PATCH)                                    |
| <code>/api/v1/events/</code>                        | GET, POST               | List events; create (admin/ministry leader)                                   |
| <code>/api/v1/events/{id}/register/</code>          | POST                    | Register/waitlist for an event                                                |
| <code>/api/v1/news/</code>                          | GET, POST               | Announcements feed, filter by <code>?is_emergency=true</code>                 |
| <code>/api/v1/gallery/albums/</code>                | GET, POST               | Gallery albums and items                                                      |
| <code>/api/v1/homilies/</code>                      | GET, POST               | Filter by <code>?priest=</code> , <code>?date=</code> , <code>?gospel=</code> |
| <code>/api/v1/readings/today/</code>                | GET                     | Cached daily readings + saint of the day                                      |
| <code>/api/v1/prayer-requests/</code>               | GET (public only), POST | Submit and view the public prayer wall                                        |
| <code>/api/v1/donations/initiate/</code>            | POST                    | Starts M-Pesa STK Push or card charge                                         |
| <code>/api/v1/donations/webhook/mpesa/</code>       | POST                    | Payment provider callback (signature verified)                                |
| <code>/api/v1/ministries/{id}/join/</code>          | POST                    | Submit a membership request                                                   |
| <code>/api/v1/search/?q=</code>                     | GET                     | Cross-model search endpoint                                                   |
| <code>/api/v1/notifications/preferences/</code>     | GET, PATCH              | Per-user channel/category opt-in settings                                     |

**Convention:** list endpoints are paginated (page size 20, cursor pagination on high-traffic ones like News and Gallery), every write endpoint enforces permission classes tied to the Role model, and all monetary/webhook endpoints are idempotent using a stored provider reference to prevent double-processing.

## Authentication, Roles & Permissions

| Role                  | Can do                                                                                                                              |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Visitor (anonymous)   | Browse public pages, submit public prayer requests, view live stream                                                                |
| Parishioner           | All Visitor abilities + register for sacraments/events, give online, join ministries, manage own profile & notification preferences |
| Ministry Leader       | All Parishioner abilities + manage own ministry's members, activities, and announcements                                            |
| Clergy                | Publish homilies/news, review sacrament registrations, respond to prayer requests                                                   |
| Admin (Parish Office) | Full access: users, donations, events, gallery, Mass schedule, all registrations                                                    |

- JWT access + refresh tokens (SimpleJWT), short-lived access token (15 min), rotating refresh token (7 days) with blacklist-on-logout
- Django's permission classes combined with a custom `IsRole("admin")`-style decorator for clean, testable per-view checks
- Social login (Google) offered as a convenience option alongside email/password for parishioners
- Password reset and email verification flows using Django's built-in token generators

# Admin Dashboard & Member Portal

---

## Admin Dashboard

### Content

Manage events, news, gallery uploads, homilies, Mass schedule, and FAQs — all via a custom React admin UI (not just the default Django admin), so non-technical office staff have a friendly interface.

### People

Manage priests/staff profiles, ministries, and user roles; approve or reject ministry membership requests.

### Requests & Registrations

Review sacrament registrations and event sign-ups; view and (where public) respond to prayer requests.

### Finance

View donations by fund/date/method, export CSV for parish accounts, monitor recurring pledges and failed payments.

**Parish statistics widgets:** weekly Mass attendance trend (if check-in is used), giving totals by fund, registration volume by sacrament, and most-viewed news/events — giving the parish office a genuine operational dashboard, not just a CMS.

## Member Portal

- View status of registered sacraments and download certificates once completed
- Register for events; see a personal "My Upcoming Events" list with .ics export
- View giving history and download an annual giving statement
- Update profile, household members, and notification preferences
- See ministries joined and pending membership requests

## Security & Data Protection

---

- ✓ HTTPS everywhere (HSTS enabled), secure/HttpOnly cookies for refresh tokens
- ✓ Django's CSRF protection on any cookie-based flow; CORS locked to known frontend origins
- ✓ Rate limiting on auth, donation, and prayer-request endpoints (django-ratelimit) to blunt brute-force and spam
- ✓ Payment webhook signatures verified server-side before marking any donation as paid
- ✓ Sensitive documents (sacrament ID uploads) stored in a private bucket with signed, expiring URLs — never public
- ✓ Role-based field-level protections: e.g. private prayer requests are excluded from every non-admin serializer at the query level, not just hidden in the UI
- ✓ Regular dependency scanning (GitHub Dependabot) and Django security-release monitoring
- ✓ Audit log of admin actions on sensitive models (donations, user roles, registration status changes)
- ✓ Personal data handling aligned with Kenya's Data Protection Act, 2019 — clear privacy policy, consent for marketing communications, and a data-export/delete path for members who request it

# Accessibility & Internationalization

---

## Accessibility (WCAG 2.1 AA target)

- High-contrast mode and user-adjustable text size, both implemented as Tailwind theme toggles rather than a separate stylesheet
- Full keyboard navigation and visible focus states on every interactive element
- Screen-reader support: semantic HTML, ARIA labels on icon-only buttons, and skip-to-content links
- Alt text required (enforced at the CMS level) for every uploaded image
- Captions/transcripts for homily videos where available

## Internationalization

- Multi-language support for English and Swahili using Django's `i18n` framework on the backend and `react-i18next` on the frontend
- Language toggle persisted per user/session; all admin-entered content (news, events, FAQs) supports optional translated fields
- Date/number formatting localized appropriately for each language



# Performance, SEO & Progressive Web App

---

## Performance

- Vite code-splitting per route; heavy pages (Gallery, Calendar) lazy-loaded
- Responsive, lazy-loaded images via Cloudinary transformations (auto format/quality)
- API responses cached where safe (daily readings, Mass schedule) with Redis + short TTLs
- Target Core Web Vitals: LCP < 2.5s, CLS < 0.1, INP < 200ms on 3G-equivalent connections common in the region

## SEO

- Server-rendered meta tags per page (via a light SSR/pre-render step or React Helmet + prerendering) so search engines and social previews see real content
- Structured data (schema.org Church, Event) for rich results on Mass times and events
- Auto-generated sitemap.xml and robots.txt; canonical URLs for paginated news/gallery

## Progressive Web App

- Installable PWA (manifest + service worker) so parishioners can add the site to their home screen like an app
- Offline fallback page and cached Mass schedule for low-connectivity moments
- Push notifications via the same PWA layer as a lightweight alternative to a native app

# Testing Strategy

| Layer                    | Tooling                                         | Coverage focus                                                                                       |
|--------------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Backend unit/integration | pytest + pytest-django + factory_boy            | Models, serializers, permission classes, registration/status workflows                               |
| API contract             | DRF test client + drf-spectacular schema checks | Every endpoint's request/response shape stays in sync with docs                                      |
| Frontend unit            | React Testing Library + Vitest                  | Form validation, role-based UI rendering, cart/registration flows                                    |
| End-to-end               | Playwright                                      | Critical paths: sacrament registration, event sign-up, donation flow, admin approving a registration |
| Payments (sandbox)       | M-Pesa/Paystack sandbox credentials             | Full donation flow incl. webhook handling, without touching real money                               |
| Accessibility            | axe-core in CI + manual screen-reader pass      | Automated regression on WCAG violations for key pages                                                |

**Target:** 80%+ coverage on backend business logic (registrations, donations, notifications), with E2E tests gating any deploy to production via CI.

# DevOps, CI/CD & Deployment

---

## Environments

- **Local:** Docker Compose (web, worker, Postgres, Redis) for parity with production
- **Staging:** auto-deployed from the develop branch for parish office review before go-live changes
- **Production:** deployed from main, behind Nginx with Gunicorn workers, on Railway/Render or a modest VPS

## CI/CD pipeline (GitHub Actions)

```
on: [push, pull_request]
jobs:
  test:
    steps:
      - checkout
      - setup Python + Node
      - pip install -r requirements.txt / npm ci
      - run ruff/black --check, eslint --max-warnings=0
      - run pytest --cov, npm run test
  build-and-deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    steps:
      - build Docker images (web, worker)
      - push to registry
      - deploy via Railway/Render CLI or SSH + docker compose pull && up -d
      - run python manage.py migrate --noinput
```

- Zero-downtime deploys via rolling restart of Gunicorn workers behind Nginx
- Automated nightly PostgreSQL backups to S3/Backblaze with a tested restore runbook
- Sentry (or self-hosted GlitchTip) for error tracking on both backend and frontend
- Uptime monitoring (e.g. UptimeRobot) on the public site and the live-stream badge endpoint

## Advanced & Differentiating Features

---

Beyond the original brief, these push the project from "church brochure site" to a genuine portfolio centerpiece.

### **Saint of the Day & Liturgical Calendar**

Auto-updating widget with season, color, and feast day — small dataset, high daily-return value.

### **QR Check-In & Giving**

QR codes for event check-in and fund-specific donations, printable on bulletins and posters.

### **Prayer Intention Wall**

Moderated public wall with a lightweight "praying for you" tap — community without becoming a social feed.

### **AI Parish Chatbot**

A scoped FAQ chatbot (RAG over the FAQ/Sacraments/Mass-times content only) answering "When is confession?" or "How do I book a wedding?" instantly, with a clear hand-off to the contact form for anything it can't answer.

### **Dark Mode**

Native Tailwind dark : variant — no separate stylesheet, toggled and remembered per user.

### **Multi-language (EN/SW)**

Full i18n rather than a translated-once landing page.

### **Role-Based Everything**

Admin, Clergy, Ministry Leader, Parishioner, Visitor — each sees a UI scoped to what they can actually do.

### **PWA + Push**

Installable, offline-tolerant, with push reminders — most of a native app's value without an app store submission.

### **Recurring Giving**

Standing pledges with automated reminders, not just one-off donations.

### **Certificate Generation**

ReportLab-generated Baptism/Confirmation certificate PDFs, downloadable from the member portal on completion.

### **Analytics Dashboard**

Parish-office-facing stats: attendance trend, giving by fund, most-registered sacraments — real operational insight, not vanity metrics.

### **Volunteer Scheduling**

Ministry leaders can post open serving slots (ushering, choir, altar serving) that members sign up for directly.

## Suggested Folder Structure

```

parish-platform/
├── backend/
│   ├── config/          # settings, urls, celery.py, asgi/wsgi
│   ├── apps/
│   │   ├── accounts/  clergy/  liturgy/  sacraments/
│   │   ├── events/    news/    media/    streaming/
│   │   ├── prayer/    giving/  ministries/  notifications/
│   │   └── search/
│   ├── tests/
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── components/ # ClergyCard, EventCard, MassTimeStrip, LiveBadge...
│   │   ├── pages/      # Home, About, MassSchedule, Sacraments/*, ...
│   │   ├── portal/     # Member portal views
│   │   ├── admin/      # Custom admin dashboard views
│   │   ├── hooks/ lib/ i18n/ styles/
│   │   └── App.jsx
│   ├── tailwind.config.js
│   ├── vite.config.js
│   └── Dockerfile
├── docker-compose.yml
└── .github/workflows/ci.yml

```

# Delivery Roadmap & Sprint Plan

A realistic phased plan for a student-led build, sized so each phase ships something demoable.

| Phase   | Focus                     | Key deliverables                                                                                       |
|---------|---------------------------|--------------------------------------------------------------------------------------------------------|
| Phase 1 | Foundations (1–2 wks)     | Project scaffolding, auth, custom User/Role model, design system in Tailwind config, homepage skeleton |
| Phase 2 | Core content (2–3 wks)    | About, Clergy, Mass Schedule, News, Events, Gallery — read-only public pages wired to the API          |
| Phase 3 | Registrations (2 wks)     | Sacrament registration workflow, event sign-up, ministry membership requests, admin approval flows     |
| Phase 4 | Money & messaging (2 wks) | M-Pesa/Paystack integration, giving history, email/SMS notifications, prayer wall                      |
| Phase 5 | Portals (1–2 wks)         | Member portal, custom admin dashboard, parish statistics widgets                                       |
| Phase 6 | Polish (1–2 wks)          | Dark mode, i18n (EN/SW), accessibility pass, PWA, SEO, performance tuning                              |
| Phase 7 | Hardening & launch        | Full test suite, security review, staging UAT with the parish office, production deploy                |



## Closing Notes

---

This specification turns a straightforward "church website" brief into a full-stack engineering project that exercises authentication and role-based access control, payment integration, real-time features, background processing, search, accessibility, and a proper CI/CD pipeline — the exact breadth of skills a Django + React portfolio project should demonstrate. Because every section maps directly to a Django app or a React route, it can be picked up phase by phase without redesigning the architecture midway, and the result is a platform built specifically for **St. Michael Madende Parish** rather than a generic template.

Built as a deliverable for **St. Michael Madende Parish**, via **jnolly IT Solutions** — reflecting the same design language and engineering rigor used across prior client and academic projects.

---

St. Michael Madende Parish — Website & Member Portal, Full-Stack Specification, Version 2.0 (Tailwind Edition). Prepared by Japeth Anold Dindi.